TRƯỜNG ĐẠI HỌC VĂN LANG

KHOA CÔNG NGHỆ THÔNG TIN

ĐỀ BÀI —BÀI TẬP 2: HIỆN THỰC HÓA MODULE CỐT LÕI

Chuyên đề tốt nghiệp 1	Track: AI
Mã luồng: L10.	Tên luồng:Tự động phân loại yêu cầu bảo hành.

Link GitHub: https://github.com/NguyenHuynhAnhTu/smartcrm-2374802010536-L10-PhanLoaiYeuCauBaoHanh

Ngày nộp: 30/10/2026

Tag phiên bản: v0.2

SVTH: Nguyễn Huỳnh Anh Tú

MSSV: 2374802010536

GVHD: ThS. Nguyễn Minh Tân

TP. Hồ Chí Minh – Năm 2026

Mục lục

MỤC 1: TÓM TẮT HỆ THỐNG 1
MỤC 2: KIẾN TRÚC ĐÃ HIỆN THỰC 2
MỤC 3: MODULE CỐT LÕI 3
MỤC 4: BỘ KIỂM THỬ 4
MỤC 5: TRÌNH BÀY KẾT QUẢ (ĐỦ 3 TRẠNG THÁI & ĐỌC CONFUSION MATRIX) 5
MỤC 6: CÁCH DỰNG LẠI HỆ THỐNG 8
MỤC 7: THAY ĐỔI SO VỚI BÀI TẬP 1 9
PHỤ LỤC: BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI 11

MỤC 1: TÓM TẮT HỆ THỐNG

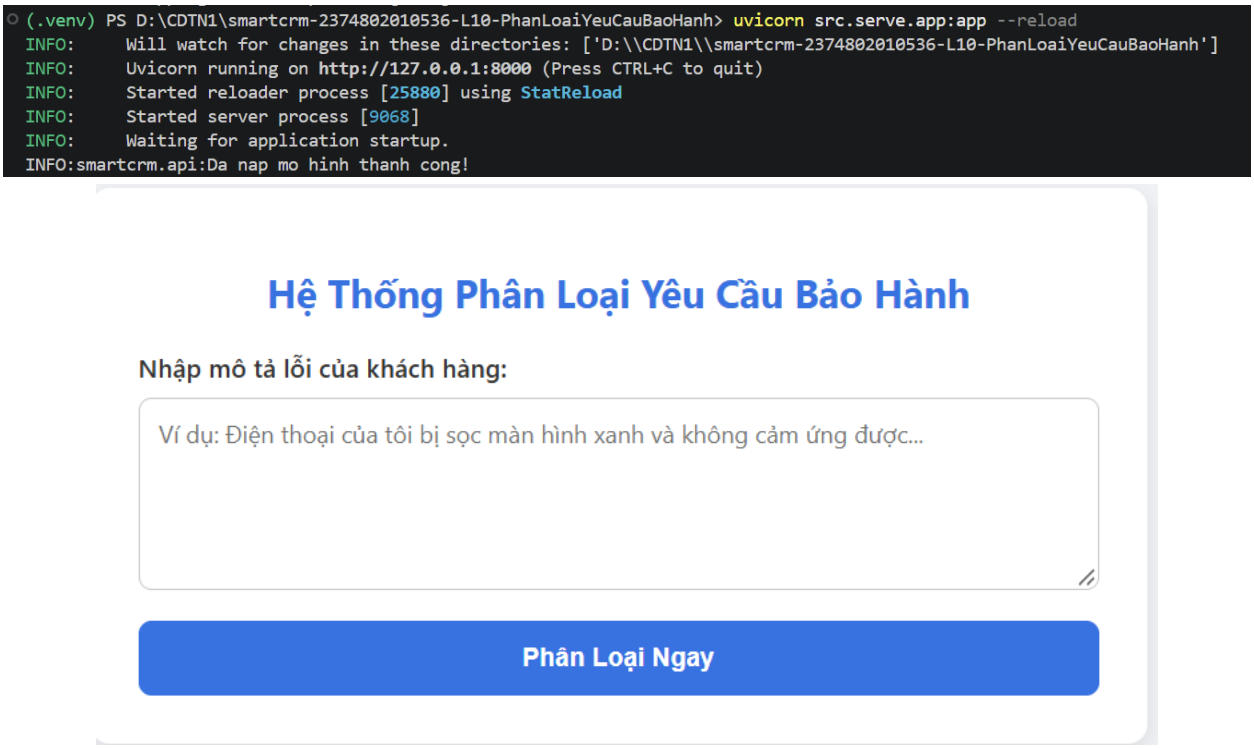
Hệ thống Smart CRM L10 được xây dựng nhằm mục đích tự động hóa quy trình phân loại yêu cầu bảo hành thiết bị. Trong phạm vi bài tập này (Track AI), hệ thống tập trung vào việc huấn luyện mô hình Học máy (Machine Learning) để phân loại văn bản mô tả lỗi của khách hàng thành các nhóm sự cố cụ thể (như Màn hình, Pin, Sạc, Nước vào...). Hệ thống không chỉ dừng lại ở việc dự đoán nhãn mà còn cung cấp độ tin cậy (Confidence Score). Đặc biệt, hệ thống được tích hợp logic nghiệp vụ: nếu độ tin cậy dưới ngưỡng 70%, hệ thống sẽ gắn cờ can_nguoi_xem_lai = true để nhân viên kiểm tra thủ công, tránh việc AI đưa ra quyết định sai lệch gây thiệt hại.

MỤC 2: KIẾN TRÚC ĐÃ HIỆN THỰC

Hệ thống được thiết kế theo kiến trúc tách lớp rõ ràng, tuân thủ nguyên tắc C2:

- Data Layer: Đọc dữ liệu thô từ issue_descriptions.csv.
- Feature Layer: Hàm clean_text() thực hiện chuẩn hóa văn bản (xóa ký tự đặc biệt, đưa về chữ thường).
- Model Layer: Sử dụng Pipeline kết hợp TfidfVectorizer (trích xuất đặc trưng) và mô hình phân loại (LogisticRegression / ComplementNB). Script huấn luyện tự động chia tập Train/Val/Test, so sánh mô hình và lưu file model_v1.joblib.
- Serving Layer: Sử dụng FastAPI để đóng gói mô hình thành RESTful API. Giao diện web HTML đơn giản được tích hợp để người dùng có thể nhập liệu trực quan.

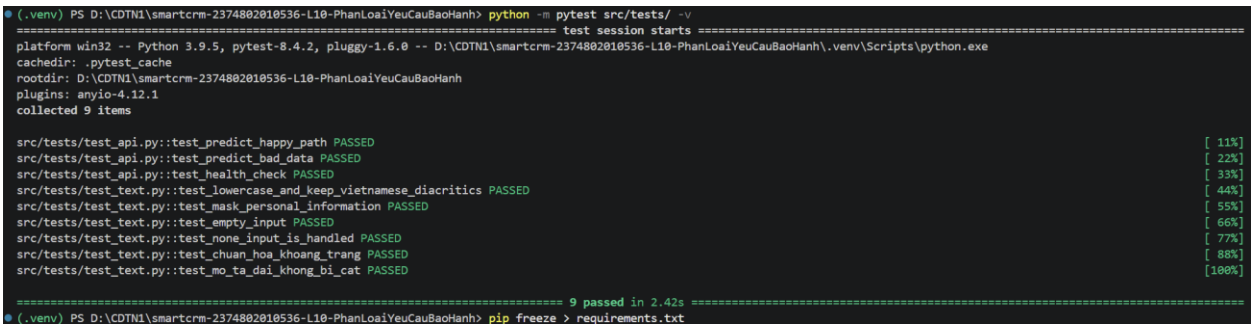
MỤC 3: MODULE CỐT LÕI



Hình 1: Hệ thống nạp thành công mô hình và thực hiện dự đoán văn bản, trả về kết quả JSON hợp lệ.

MỤC 4: BỘ KIỂM THỬ

Hệ thống đã xây dựng bộ kiểm thử tự động với 3 loại test (Luồng đúng, Biên, Dữ liệu sai). Sử dụng thư viện pytest để chạy.

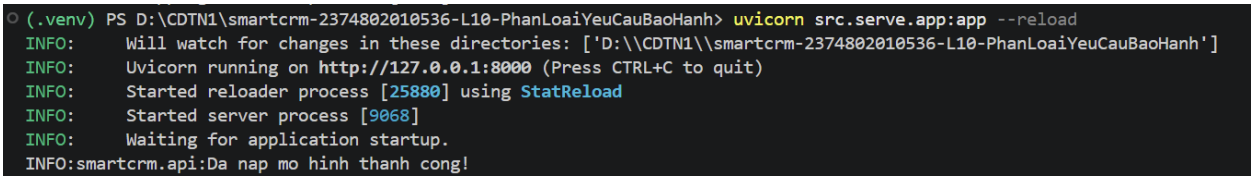


Hình 2: Kết quả chạy pytest thành công, các test case (bao gồm test API từ chối dữ liệu rỗng) đều PASS.

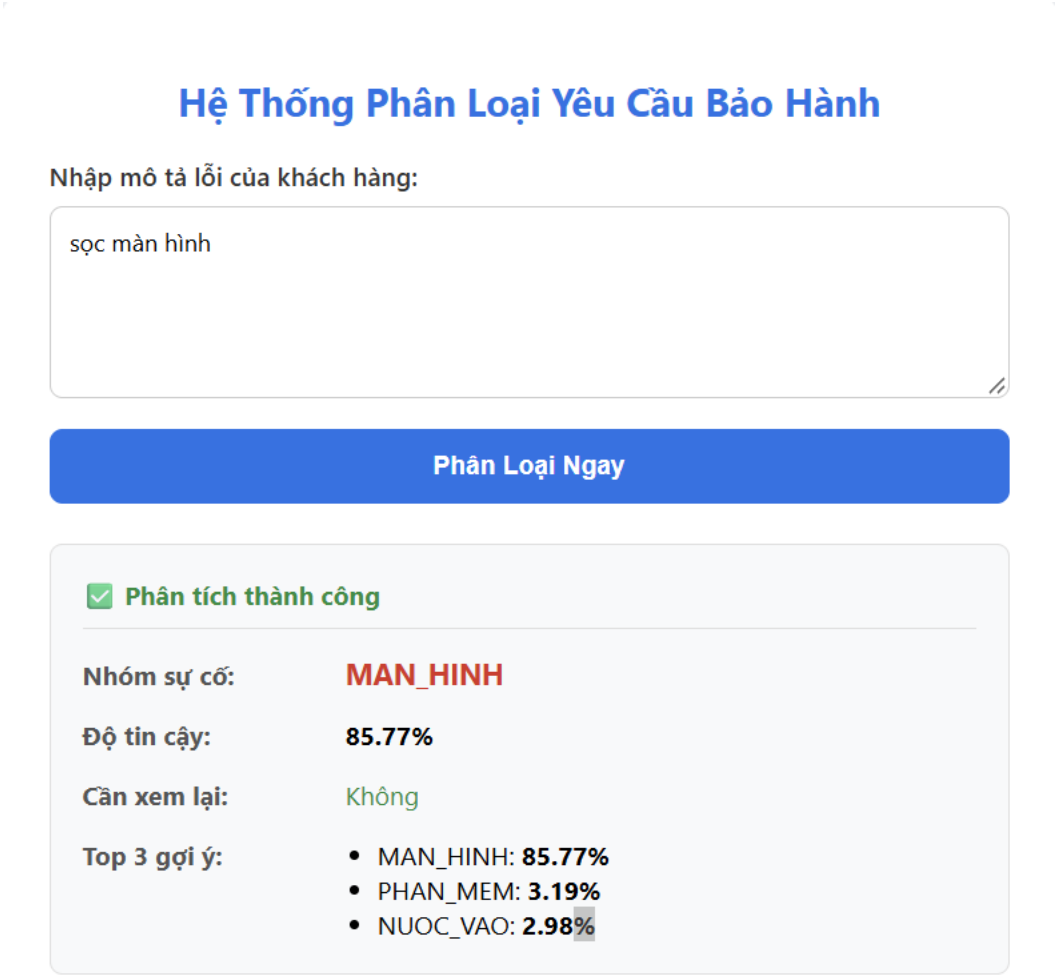
MỤC 5: TRÌNH BÀY KẾT QUẢ (ĐỦ 3 TRẠNG THÁI & ĐỌC CONFUSION MATRIX)

Hệ thống API đã được kiểm thử và xử lý đầy đủ 3 trạng thái trên giao diện người dùng:

1. Trạng thái Khởi tạo / Nạp mô hình (Loading/Initialization):
Trước khi có thể đưa ra dự đoán, hệ thống Backend (FastAPI) thực hiện quá trình nạp file mô hình (model_v1.joblib) vào RAM. Trạng thái này được ghi nhận qua log của hệ thống. Nếu nạp thất bại, API sẽ báo lỗi và dừng khởi động.

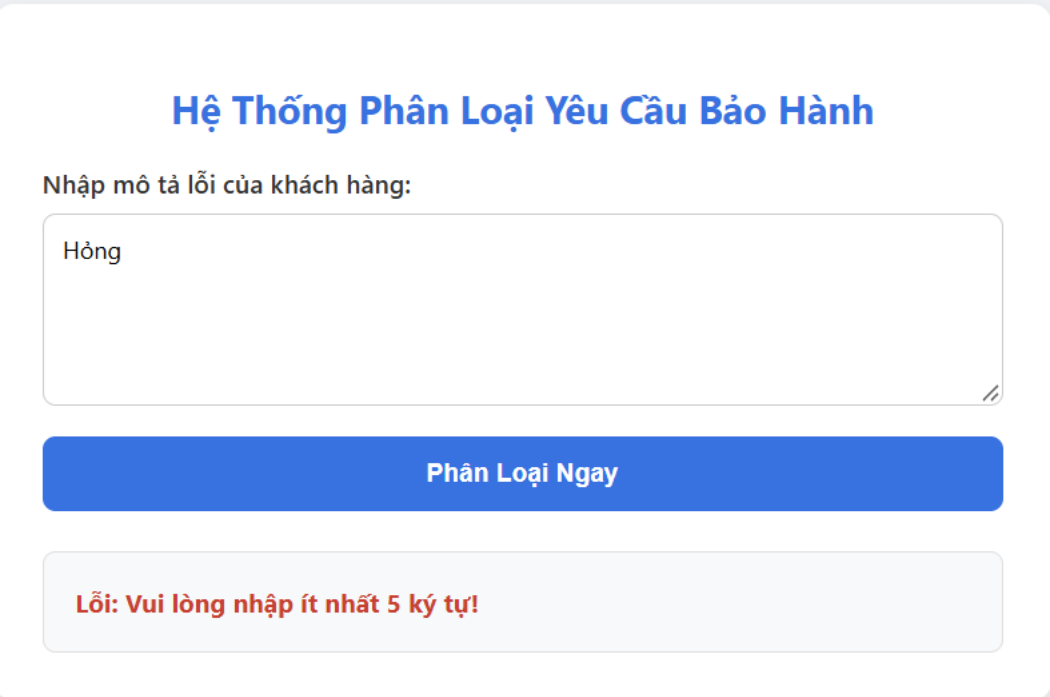


2.Trạng thái Thành công (Success):
Khi người dùng nhập đúng định dạng mô tả lỗi (trên 5 ký tự), hệ thống gọi API và hiển thị kết quả phân loại bao gồm: Nhóm sự cố dự đoán, Độ tin cậy (%) và danh sách Top 3 gợi ý.



Hình 3: Trạng thái hệ thống phân tích thành công.

3. Trạng thái Lỗi / Rỗng (Error/Empty - Xử lý dữ liệu sai C7):
Hệ thống tích hợp lớp Validation (Pydantic) để kiểm tra dữ liệu đầu vào. Nếu người dùng bỏ trống hoặc nhập mô tả quá ngắn (dưới 5 ký tự), API sẽ chặn lại và trả về mã lỗi 422 Unprocessable Entity kèm thông báo rõ ràng cho người dùng. Điều này đảm bảo hệ thống không bị crash khi gặp dữ liệu rác.



Hình 4: Trạng thái hệ thống báo lỗi

Đọc kết quả từ Confusion Matrix & Xử lý Bias (Điểm nhấn kỹ thuật):
Thông qua quá trình đánh giá mô hình (file metrics.json), em nhận thấy mô hình có hiện tượng thiên kiến dữ liệu (Class Imbalance). Cụ thể, do số lượng mẫu nhãn NUOC_VAO và MAN_HINH chiếm đa số, khi gặp các câu mô tả mơ hồ, mô hình có xu hướng dự đoán nhầm sang các nhãn này.

Giải pháp đã áp dụng: Em đã tích hợp tham số class_weight='balanced' vào quá trình huấn luyện để phạt nặng các nhãn ít mẫu, đồng thời thiết lập Ngưỡng tin cậy (Threshold) = 0.70 trên API. Nếu kết quả dự đoán có xác suất dưới 70%, API sẽ trả về cờ can_nguoi_xem_lai = true, yêu cầu nhân viên hỗ trợ thay vì tự động ra quyết định.

MỤC 6: CÁCH DỰNG LẠI HỆ THỐNG

Hệ thống có thể được dựng lại dễ dàng từ thư mục trống bằng các lệnh sau (đã ghi rõ trong README.md):

1. Clone repo: git clone --branch v0.2 <link_repo>
2. Cài đặt môi trường: python -m venv .venv -> Kích hoạt -> pip install -r requirements.txt
3. Huấn luyện lại mô hình (tùy chọn): python -m src.models.train
4. Chạy API Server: uvicorn src.serve.app:app --reload
5. Chạy test: pytest src/tests/

MỤC 7: THAY ĐỔI SO VỚI BÀI TẬP 1

Nhật ký thay đổi thiết kế

Sau phản hồi Bài tập 1

Thời điểm: Buổi 8 – Ngày 19 tháng 10 năm 2026

TD-01 – Chuẩn hóa mã Use Case và bảng truy vết yêu cầu

- Phản hồi của GV: Sơ đồ Use Case sử dụng mã US-01 đến US-08 thay vì UC01 đến UC08; bảng truy vết liên kết FR05 với US05 không đúng với nội dung User Story; đặc tả UC02 ghi nhầm User Story liên quan; US08 chưa được thể hiện trong bảng truy vết.

- Quyết định:SỬA.

- Lý do: Cần phân biệt mã User Story (US) và mã Use Case (UC), đồng thời bảo đảm các liên kết giữa yêu cầu chức năng, User Story và Use Case nhất quán trong toàn bộ tài liệu.

- Ảnh hưởng: Chuẩn hóa mã Use Case trên sơ đồ; cập nhật bảng truy vết và đặc tả Use Case. Sau khi điều chỉnh danh sách User Story, các mã và liên kết được cập nhật theo thiết kế mới gồm US01–US07 và UC01–UC07.

TD-02 – Điều chỉnh số lượng User Story

- Phản hồi của GV: Thiết kế có 8 User Story, vượt yêu cầu từ 5 đến 7 User Story.

- Quyết định: SỬA.

- Lý do: Danh sách User Story cần đáp ứng giới hạn của bài tập và tránh tách riêng những thao tác có thể được mô tả trong cùng một mục tiêu nghiệp vụ.

- Ảnh hưởng: Gộp User Story xác nhận nhóm sự cố và User Story chỉnh sửa nhóm sự cố thành US04 – Xác nhận hoặc chỉnh sửa kết quả phân loại. Các User Story phía sau được đánh lại mã: US05 – Lưu kết quả phân loại, US06 – Tra cứu yêu cầu đã phân loại và US07 – Xem thống kê nhóm sự cố. Cập nhật SRS, bảng truy vết và sơ đồ liên quan.

TD-03 – Bổ sung chú thích ký hiệu cho các sơ đồ

- Phản hồi của GV: Sơ đồ Use Case, sơ đồ kiến trúc và ML pipeline chưa có chú thích ký hiệu.

- Quyết định: SỬA.

- Lý do: Chú thích giúp người đọc hiểu ý nghĩa của các thành phần, ký hiệu và mối quan hệ được sử dụng trong từng sơ đồ.

- Ảnh hưởng: Bổ sung chú thích tác nhân, Use Case, ranh giới hệ thống và các quan hệ `include`/`extend` trên sơ đồ Use Case; bổ sung chú thích phù hợp cho sơ đồ kiến trúc và ML pipeline. Sử dụng các tệp nguồn có thể chỉnh sửa và cập nhật hình ảnh xuất ra trong thư mục `docs/`.

TD-04 – Bổ sung tiêu chí định nghĩa sáu nhãn phân loại

- Phản hồi của GV: Chưa có định nghĩa rõ ràng cho các nhãn MAN_HINH, PIN, SAC, PHAN_MEM, NUOC_VAO và KHAC.

- Quyết định: SỬA.

- Lý do: Tiêu chí gán nhãn cụ thể giúp việc phân loại dữ liệu nhất quán hơn, hỗ trợ kiểm tra chất lượng dữ liệu và đánh giá kết quả mô hình.

- Ảnh hưởng: Bổ sung mô tả, dấu hiệu nhận biết và ví dụ cho từng nhãn trong `docs/ml-problem-statement.md`. Làm rõ cách phân loại các mô tả mơ hồ hoặc có dấu hiệu thuộc nhiều nhóm; đối chiếu tiêu chí với dữ liệu thực tế trước khi áp dụng.

TD-05 – Liên kết wireframe với Use Case

- Phản hồi của GV: Wireframe chưa ghi rõ mỗi màn hình tương ứng với Use Case nào.

- Quyết định: SỬA.

- Lý do: Liên kết giữa màn hình và Use Case giúp đối chiếu giao diện với yêu cầu nghiệp vụ, đồng thời kiểm tra tính nhất quán giữa SRS, sơ đồ Use Case và thiết kế giao diện.

- Ảnh hưởng: Bổ sung mã Use Case tương ứng cho từng màn hình trong wireframe; đối chiếu lại chức năng của các màn hình với đặc tả Use Case và bảng truy vết yêu cầu.

TD-06 – Hoàn thiện cấu trúc repo và tài liệu dự án

- Phản hồi của GV: Tại thời điểm nộp BT1, repo có một commit và thiếu thư mục `docs/`, các tệp sơ đồ `.drawio` gốc, `srs.md`, `ai-disclosure.md` và `ml-problem-statement.md`.

- Quyết định: SỬA.

- Lý do: Repo cần lưu trữ đầy đủ tài liệu yêu cầu, thiết kế, tệp nguồn sơ đồ và thông tin sử dụng AI để hỗ trợ kiểm tra, chỉnh sửa và theo dõi tiến độ giữa các bài tập.

- Ảnh hưởng: Kiểm tra và hoàn thiện cấu trúc repo; bổ sung hoặc cập nhật các tài liệu còn thiếu; lưu tệp `.drawio` có thể chỉnh sửa bên cạnh hình ảnh xuất ra; cập nhật README và tạo các commit riêng theo từng nhóm công việc bằng thông điệp phù hợp.

PHỤ LỤC: BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI

STT	Công cụ AI	Nội dung công việc được AI hỗ trợ	Tệp / Hàm / Vị trí liên quan	Cách sinh viên kiểm chứng / Tự đánh giá
1	ChatGPT	Gợi ý cách xử lý bài toán mất cân bằng dữ liệu (Class Imbalance) bằng cách thêm tham số class_weight='balanced' vào mô hình Scikit-learn.	src/models/train.py	Chạy huấn luyện lại, so sánh chỉ số F1-score trong metrics.json trước và sau khi thêm tham số để thấy sự cải thiện..
2	ChatGPT	Hướng dẫn cấu trúc thư mục FastAPI, cách dùng thư viện Pydantic để validate dữ liệu và cách bắt lỗi 422 (Bad Request).	src/serve/app.py	Tự dùng Swagger UI/Postman gửi request sai (nhập dưới 5 ký tự) để xem API có chặn và trả về lỗi 422 đúng như mong đợi không.
3	ChatGPT	Gợi ý viết các test case bằng Pytest cho cả hàm tiền xử lý văn bản và API (bao gồm test luồng đúng, test dữ liệu sai, test health check).	src/tests/	Tự chạy lệnh pytest src/tests/ -v trên Terminal, kiểm tra thấy toàn bộ 9 test đều báo PASSED.
4	ChatGPT	Hỗ trợ viết mã HTML/CSS/JS cơ bản để tạo giao diện Web demo nhập liệu trực quan thay vì chỉ dùng Swagger UI.	giao_dien_demo() - app.py	Mở trình duyệt, tự tay nhập liệu, kiểm tra giao diện hiển thị kết quả JSON có đúng với thiết kế và xử lý lỗi mượt mà không.
5	ChatGPT	Gợi ý cấu trúc file README, cách viết file .gitignore chuẩn cho Python và cách đánh Tag trên Git..	README.md, .gitignore	Tự tay clone repo sang một thư mục mới, làm đúng theo hướng dẫn trong README để xác nhận hệ thống chạy được từ đầu đến cuối..

Cam kết: Tôi xác nhận đã đọc, hiểu và chịu trách nhiệm về toàn bộ nội dung nộp.

Họ tên: Nguyễn Huỳnh Anh Tú

MSSV: 2374802010536

Ngày:30/10/2026